

Guide d'Intégration Google Auth via Keycloak - NextStep

Ce guide détaille la mise en place de l'authentification Google dans la plateforme NextStep.

💡 Concept Fondamental

Dans l'architecture de NextStep, **Keycloak agit comme le Broker d'Identité**. L'intégration ne se fait pas directement dans Angular ou .NET, mais via Keycloak.

Le flux d'authentification est le suivant : **Angular** → **Keycloak** → **Google** → **Keycloak** → **Angular** → **.NET**

🔧 Étape 1 : Configuration sur la Google Cloud Console

1. Accédez à la [Google Cloud Console](#).
2. Créez un projet (ex: **NextStep-Auth**).
3. **OAuth consent screen** :
 - Type : **External**.
 - Scopes : **.../auth/userinfo.email** et **.../auth/userinfo.profile**.
 - ⚠ **Mode Testing** : Si le projet reste en mode "Testing", vous devez **ajouter manuellement chaque email de test** dans la section "Test users". Sinon Google bloquera la connexion.
4. **Credentials** → **OAuth client ID** → **Web application** :
 - **Authorized redirect URIs** : **<http://localhost:8080/realms/Next-Step/broker/google/endpoint>**

⚠ **Attention à la casse** : Le nom du realm (**Next-Step**) doit correspondre **exactement** à celui configuré dans Keycloak (majuscules et tiret inclus).

5. Récupérez le **Client ID** et le **Client Secret**.

💡 Lors du copier-coller, vérifiez qu'aucun espace caché n'est ajouté au début ou à la fin du secret.

🔑 Étape 2 : Configuration dans Keycloak 24

1. Console d'administration Keycloak → Realm : **Next-Step**.
2. **Identity Providers** → **Add provider** → **Google**.
3. Configuration (General Settings) :
 - **Alias** : **google** (doit correspondre à l'alias utilisé dans les templates du thème)
 - **Client ID** : (Copié depuis Google Cloud Console)
 - **Client Secret** : (Copié depuis Google Cloud Console)
4. Configuration (Advanced Settings) :
 - **Scopes** : **openid email profile** (⚠ **obligatoire**, sans cela Keycloak ne recevra pas les infos utilisateur)
 - **Store tokens** : **On**

- **Trust Email** : **On** (évite la demande de vérification email si SMTP n'est pas configuré)
- **First Broker Login Flow** : **first broker login** (gère la création/liaison de comptes)
- **Sync mode** : **Import**

🧐 Étape 3 : Intégration dans le Thème Keycloak (Templates FTL)

Les boutons sociaux dans le thème custom doivent utiliser les **URLs de broker générées par Keycloak** via la variable `social.providers`, et **non** des URLs construites manuellement.

✗ Ce qu'il ne faut PAS faire

```
<!-- INCORRECT : kc_idp_hint n'est pas traité dans les URLs login-actions -->
<a href="{url.loginUrl}&kc_idp_hint=google">Google</a>

<!-- INCORRECT : provoque un UnsupportedOperationException dans Keycloak 24 -->
<a href="{url.registrationAction}&kc_idp_hint=google">Google</a>
```

Pourquoi ? Le paramètre `kc_idp_hint` n'est traité qu'au début du flux OIDC (Authorization Endpoint). Les URLs `login-actions/authenticate` sont des URLs internes qui ne le reconnaissent pas. De plus, `registrationAction` envoie la requête à travers le validateur de mot de passe, ce qui crash car un login social ne fournit pas de mot de passe.

☑ La bonne méthode : utiliser `social.providers`

```
<!-- login.ftl & register.ftl -->
<#if social?? && social.providers??>
  <div class="social-providers">
    <#list social.providers as p>
      <#if p.alias == "google">
        <a href="{p.loginUrl}" class="btn-social" id="social-google">
          <!-- SVG Google icon -->
          Google
        </a>
      <#elseif p.alias == "github">
        <a href="{p.loginUrl}" class="btn-social" id="social-github">
          <!-- SVG GitHub icon -->
          GitHub
        </a>
      </#if>
    </#list>
  </div>
</#if>
```

Pourquoi ça marche ? `{p.loginUrl}` contient l'URL complète du broker Keycloak avec les tokens de session, le `tab_id` et le `session_code` nécessaires pour initier correctement le flux Identity Provider.

Fallback pour la page d'inscription

Si `social.providers` n'est pas disponible dans le contexte de la page d'inscription, utiliser un lien direct vers l'endpoint OIDC :

```
<a href="/realms/Next-Step/protocol/openid-connect/auth?client_id=nextstep-frontend&redirect_uri=http%3A%2F%2Flocalhost%3A4200%2F&response_type=code&scope=openid&kc_idp_hint=google">
  Google
</a>
```

Étape 4 : Service Angular (Frontend)

Un `AuthService` centralisé permet de déclencher les connexions sociales depuis le code Angular si nécessaire (ex: depuis un bouton dans l'application).

Fichier : `src/app/core/auth/services/auth.service.ts`

```
import { Injectable, inject } from '@angular/core';
import Keycloak from 'keycloak-js';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly keycloak = inject(Keycloak);

  // Connexion standard (affiche la page Keycloak)
  login() {
    return this.keycloak.login();
  }

  // Connexion directe via Google (bypass la page de sélection)
  loginWithGoogle() {
    return this.keycloak.login({
      idpHint: 'google' // ⚠ "idpHint" pour keycloak-js v26+
    });
  }

  // Connexion directe via GitHub
  loginWithGithub() {
    return this.keycloak.login({
      idpHint: 'github'
    });
  }

  logout() {
    return this.keycloak.logout({
      redirectUri: window.location.origin
    });
  }
}
```

⚠ **Attention au nom du paramètre** : Dans `keycloak-js` v26+, le paramètre s'appelle `idpHint` (camelCase). L'ancien nom `kcIdentityProvider` (utilisé dans certaines versions de `keycloak-angular`) est **déprécié** et provoquera une erreur TypeScript.

L'intercepteur `includeBearerTokenInterceptor` (configuré dans `app.config.ts`) continue d'injecter automatiquement le Bearer Token JWT dans chaque requête vers le backend.

⚙ Étape 5 : Validation Backend (.NET 8)

Aucune modification n'est nécessaire.

Le backend utilise le **JIT Provisioning** (déjà implémenté dans `UserService.cs`). À chaque token validé, il :

- 1. Extrait le `sub` (Keycloak ID), `email`, `given_name`, `family_name` depuis les claims JWT.
- 2. Vérifie si l'utilisateur existe dans la table `utilisateurs`.
- 3. Crée automatiquement le profil si c'est la première connexion.

Le backend reste **indépendant du fournisseur d'identité** (Google, GitHub, email/password). Il ne fait confiance qu'à la signature du JWT émis par Keycloak.

📋 Résumé du Flux de Données

- 1. **Angular** → L'utilisateur arrive sur l'app, le `authGuard` redirige vers Keycloak.
- 2. **Keycloak** → Affiche la page de login avec les boutons sociaux (Google/GitHub).
- 3. **Utilisateur** → Clique sur Google → Keycloak redirige vers Google OAuth2.
- 4. **Google** → Authentifie l'utilisateur, renvoie le code d'autorisation à Keycloak.
- 5. **Keycloak** → Crée/lie l'utilisateur local, génère un **JWT** avec les claims.
- 6. **Angular** → Reçoit le JWT, l'injecte dans les requêtes vers le backend .NET.
- 7. **.NET** → Valide le JWT, exécute le JIT Provisioning → Accès autorisé.

⚠ Points de Vigilance

Sujet	Détail
HTTPS	Obligatoire en production pour les redirections OAuth2.
Redirect URI	Doit correspondre exactement (casse incluse) entre Google Console et Keycloak.
Scopes Keycloak	Le champ "Scopes" dans Advanced Settings ne doit pas être vide — mettre <code>openid email profile</code> .
Trust Email	Activer pour éviter le blocage si SMTP n'est pas configuré.
Test Users	En mode "Testing" de Google, ajouter chaque email de test manuellement.
Client Secret	Vérifier qu'aucun espace caché n'est copié lors du collage.
Nom du paramètre JS	<code>idpHint</code> pour keycloak-js v26+, pas <code>kcIdentityProvider</code> .

Sujet	Détail
URLs dans le thème	Utiliser <code>\${p.loginUrl}</code> de <code>social.providers</code> , jamais <code>\${url.loginUrl}&kc_idp_hint=...</code>
Bug Keycloak 24	<code>\${url.registrationAction}&kc_idp_hint=...</code> provoque un crash (<code>EmptyMultivaluedMap.remove()</code>).

🐛 Problèmes Connus & Solutions

"Unexpected error when handling authentication request to identity provider"

Cause : Le bouton social utilise `${url.registrationAction}` au lieu de `${p.loginUrl}`. **Solution** : Utiliser `social.providers` dans les templates FTL (voir Étape 3).

Le bouton Google ne fait rien (redirige vers la page login sans action)

Cause : L'URL utilise `${url.loginUrl}&kc_idp_hint=google`. Ce paramètre n'est pas traité dans les URLs `login-actions`. **Solution** : Utiliser `${p.loginUrl}` depuis `social.providers`.

`invalid_user_credentials` dans les logs avec `register_method="form"`

Cause : Bug dans Keycloak 24. `RegistrationPassword.validate()` essaie d'appeler `remove()` sur une `EmptyMultivaluedMap` (immutable) car le login social ne fournit pas de mot de passe. **Solution** : Ne jamais envoyer un login social à travers `${url.registrationAction}`.

Erreur TypeScript `kcIdentityProvider` non reconnu

Cause : `keycloak-js` v26+ utilise `idpHint` au lieu de `kcIdentityProvider`. **Solution** : Remplacer par `idpHint: 'google'` dans le `AuthService`.